

Importance of Secure Code

What are Buffer Overflows?

- ▶ Occurs when a program writes data beyond the allocated buffer
- ▶ One of the oldest and most common security vulnerabilities
- ▶ Still relevant in 2025 despite modern protections
- ▶ Can lead to:
 - ▶ Program crashes
 - ▶ Arbitrary code execution
 - ▶ Privilege escalation

Why Should You Care?

- ▶ MITRE's CWE-119 (Improper Restriction of Operations within the Bounds of a Memory Buffer)
 - ▶ Consistently in the top 25 most dangerous software weaknesses
- ▶ Understanding buffer overflows helps you:
 - ▶ Write more secure code
 - ▶ Identify vulnerabilities in existing code
 - ▶ Understand fundamental memory safety concepts

User Space Buffer Overflows

Memory Layout Basics

- ▶ Process memory layout:
 - ▶ Stack (local variables, function calls)
 - ▶ Heap (dynamically allocated memory)
 - ▶ Data (initialized/uninitialized variables)
 - ▶ Text (executable code)
- ▶ Stack grows downward (from high to low addresses)
- ▶ Each function call creates a new stack frame containing:
 - ▶ Function parameters
 - ▶ Return address
 - ▶ Saved frame pointer
 - ▶ Local variables

Stack-Based Buffer Overflow: The Mechanics

- ▶ Local buffer variables are stored on the stack
- ▶ No automatic bounds checking in C/C++
- ▶ Overwriting beyond buffer boundaries can corrupt:
 - ▶ Other local variables
 - ▶ Saved frame pointer
 - ▶ Return address
 - ▶ Function parameters

Demonstration: A Vulnerable Program

```
#include <stdio.h>
#include <string.h>
void function(char *input) {
    char buffer[64];
    strcpy(buffer, input);
    printf("Input was: %s\n", buffer);
}
void spawn_shell() {
    char *args[] = {"/bin/sh", NULL};
    execve("/bin/sh", args, NULL);
}

int main(int argc, char *argv[]) {
    if (argc > 1)
        function(argv[1]);
    return 0;
}
```

Demonstration: A Vulnerable Program

```
#include <stdio.h>
#include <string.h>
void function(char *input) {
    char buffer[64];
    strcpy(buffer, input); //no bounds checking!
    printf("Input was: %s\n", buffer);
}
void spawn_shell() {
    char *args[] = {"/bin/sh", NULL};
    execve("/bin/sh", args, NULL);
}

int main(int argc, char *argv[]) {
    if (argc > 1)
        function(argv[1]);
    return 0;
}
```

Exploitation Step 1: Crash It

- ▶ Compile without protections:

```
gcc -fno-stack-protector -z execstack \  
-o vulnerable vulnerable.c
```

- ▶ Crash the program with a large input:

```
./vulnerable $(python -c 'print("A" * 100)')
```

- ▶ Segmentation fault occurs when the return address is corrupted
- ▶ Use debugging tools (GDB) to analyze the crash

Exploitation Step 2: Control the Execution

- Determine the exact offset to the return address:

Create a pattern

cyclic 100

Run with the pattern and find the offset

cyclic -l <value in EIP/RIP register>

- Crafting the exploit:

1. Fill the buffer (64 bytes)
2. Overwrite return address with desired address

Exploitation Step 3: Exploiting

- ▶ Basic exploit structure:

[Padding][New Return Address]

- ▶ Simple demonstration exploit:

```
./vulnerable $(python -c 'print( "A" * <padding>\n\n+ "<return_address>")')
```

Mitigations: User Space

- ▶ Address Space Layout Randomization (ASLR)
 - ▶ Randomizes memory addresses
- ▶ Stack canaries
 - ▶ Places known values before return addresses
 - ▶ Checked before function returns
- ▶ Data Execution Prevention (DEP)/NX bit
 - ▶ Prevents executing code on the stack
- ▶ Position Independent Executables (PIE)
 - ▶ Code compiled to use relative addressing
- ▶ Control Flow Integrity (CFI)
 - ▶ Ensures program follows a legitimate control flow path

Kernel Space Buffer Overflows

Kernel Memory Layout

- ▶ Kernel space vs. User space memory
- ▶ Kernel components:
 - ▶ Core kernel code
 - ▶ Loadable kernel modules
 - ▶ Kernel stacks
 - ▶ Heap (kmalloc)
- ▶ Different memory allocation mechanisms
 - ▶ kmalloc, vmalloc, etc.
- ▶ Different protection mechanisms

Why Kernel Vulnerabilities Matter

- ▶ Impact of kernel exploitation:
 - ▶ Complete system compromise
 - ▶ Persistence across user sessions
 - ▶ Access to all system resources
 - ▶ Bypass of user space protections
- ▶ Attack surfaces:
 - ▶ System calls
 - ▶ Device drivers
 - ▶ File systems
 - ▶ Network stack

Vulnerable Kernel Module Example - Part 1

```
// vulnerable_module.c
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/proc_fs.h>
#include <linux/string.h>
#include <linux/uaccess.h>

MODULE_LICENSE("GPL");

#define PROC_ENTRY "buffer_overflow"
#define BUFFER_SIZE 64

static struct proc_dir_entry *proc_entry;
static char kernel_buffer[BUFFER_SIZE];
```

Vulnerable Kernel Module Example - Part 2

```
static ssize_t device_write(struct file *file,
                           const char __user *buffer,
                           size_t len, loff_t *offset) {
    // Vulnerable: no bounds checking!
    if (copy_from_user(kernel_buffer, buffer, len)) {
        return -EFAULT;
    }

    printk(KERN_INFO "Received data of length %ld\n", len);
    return len;
}

static const struct proc_ops proc_ops = {
    .proc_write = device_write,
};
```

Vulnerable Kernel Module Example - Part 3

```
static int __init vulnerable_init(void) {
    proc_entry = proc_create(PROC_ENTRY, 0666,
                             NULL, &proc_ops);

    if (!proc_entry) {
        return -ENOMEM;
    }
    printk(KERN_INFO "Vulnerable module loaded\n");
    return 0;
}

static void __exit vulnerable_exit(void) {
    proc_remove(proc_entry);
    printk(KERN_INFO "Vulnerable module unloaded\n");
}

module_init(vulnerable_init);
module_exit(vulnerable_exit);
```

Kernel Exploitation Approach

► Exploitation steps:

1. Trigger the buffer overflow
2. Overwrite kernel data structures
3. Redirect control flow
4. Execute privileged operations

► Example trigger:

```
echo -n "$(python -c 'print("A" * 100)')" \  
> /proc/buffer_overflow
```

► Target for exploitation:

- Function pointers
- Return addresses
- Critical kernel structures

Kernel Exploit Payload

▶ Common payloads:

```
// Privilege escalation payload  
#include <sys/types.h>  
  
int payload(void) {  
    // Find current process cred structure  
    commit_creds(prepare_kernel_cred(0));  
    // Return to user space  
    return 0;  
}
```

▶ Privilege escalation techniques:

- ▶ Modifying cred structure (UID/GID)
- ▶ Disabling security mechanisms
- ▶ Installing backdoors

Kernel Protection Mechanisms

- ▶ KASLR (Kernel Address Space Layout Randomization)
- ▶ SMEP/SMAP (Supervisor Mode Execution/Access Prevention)
- ▶ KPTI (Kernel Page Table Isolation)
- ▶ Control Flow Integrity
- ▶ Restricted access to /dev and /proc interfaces
- ▶ CONFIG_HARDENED_USERCOPY
- ▶ Memory tagging

Secure Coding Practices

Safer Programming Languages

- ▶ Memory-safe languages:
 - ▶ Rust
 - ▶ Go
 - ▶ Java
 - ▶ Python
 - ▶ C# (.NET)
- ▶ When using C/C++:
 - ▶ Modern C++ practices
 - ▶ Smart pointers
 - ▶ Standard containers (`std::vector`, `std::string`)

Secure C/C++ Coding

- ▶ Use bound-checking functions:
 - ▶ `strncpy()` instead of `strcpy()`
 - ▶ `strncat()` instead of `strcat()`
 - ▶ `snprintf()` instead of `sprintf()`
- ▶ Validate all inputs:
 - ▶ Length checks
 - ▶ Format validation
 - ▶ Type checking
- ▶ Memory management best practices:
 - ▶ Free all allocated memory
 - ▶ Check for successful allocations
 - ▶ Avoid double free issues

Secure Function Use Examples

// Unsafe

```
char buffer[64];  
strcpy(buffer, user_input); // No bounds checking!
```

// Safer

```
char buffer[64];  
strncpy(buffer, user_input, sizeof(buffer) - 1);  
buffer[sizeof(buffer) - 1] = '\\0'; // Ensure null termination
```

// Even better

```
char buffer[64];  
snprintf(buffer, sizeof(buffer), "%s", user_input);
```

Static Analysis Tools

- ▶ CodeQL
 - ▶ Query-based analysis
 - ▶ Can find complex vulnerability patterns
 - ▶ Used by GitHub Security Lab
- ▶ Clang Static Analyzer
 - ▶ Built into the Clang compiler
 - ▶ Path-sensitive analysis

CodeQL

```
import cpp

// Find functions that use unsafe functions like strcpy
from FunctionCall call
where call.getTarget().hasName("strcpy")
select call, "Potential buffer overflow detected."
```

Demo

Fuzzing: Finding Vulnerabilities Automatically

- ▶ Fuzzing: sending random/semi-random inputs to find crashes
- ▶ Types of fuzzers:
 - ▶ Mutation-based (AFL, AFL++)
 - ▶ Generation-based (Peach)
 - ▶ Coverage-guided (libFuzzer)
- ▶ Example with AFL++:

Compile with instrumentation

```
afl-gcc -o target target.c
```

Run the fuzzer

```
afl-fuzz -i input_dir -o output_dir ./target @@
```

Demo

Runtime Protection Mechanisms

- ▶ Enable all compiler protections:

```
gcc -Wall -Wextra -Werror -fstack-protector-all \
    -pie -fPIE -D_FORTIFY_SOURCE=2 -O2 -Wl,-z,relro,-z,m
```

- ▶ Tools for hardening:

- ▶ AddressSanitizer (ASan)
- ▶ UndefinedBehaviorSanitizer (UBSan)
- ▶ ThreadSanitizer (TSan)

- ▶ Seccomp filters for system call restrictions
- ▶ AppArmor/SELinux profiles

Conclusion

Key Takeaways

- ▶ Buffer overflows remain a critical security concern
- ▶ Understanding memory layout is essential for secure programming
- ▶ Different environments (user space vs. kernel) require different approaches
- ▶ Defense in depth:
 - ▶ Use memory-safe languages when possible
 - ▶ Apply secure coding practices
 - ▶ Utilize static analysis and fuzzing
 - ▶ Enable runtime protections
 - ▶ Regular security testing

Questions?